



Universidad Nacional de Río Cuarto

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Simulación – Prof. Ariel González

Proyecto de Simulación

Sistema de Tiempo Real: Controlador Simplificado de Bomba
de Infusión

Simulación de Sistemas

Autor:

Emiliano Bernal

Río Cuarto, Córdoba
Junio de 2026

Índice

1. Descripción general	2
2. Objetivo del proyecto	2
3. Entradas del sistema	2
4. Salidas del sistema	3
5. Requisitos temporales	3
6. Parámetros del sistema	4
7. Componentes propuestos	4
7.1. Generador de órdenes médicas	4
7.2. Sensor de flujo	4
7.3. Controlador de bomba	4
7.4. Actuador de la bomba	5
7.5. Módulo de alarmas	5
8. Modelos atómicos DEVS	5
8.1. Generador de órdenes médicas	5
8.2. Sensor de flujo	6
8.3. Controlador de bomba	7
8.4. Actuador de la bomba	12
8.5. Módulo de alarmas	13
8.6. Registro de eventos	15
9. Modelo acoplado	18
9.1. Especificación formal del acoplamiento	18

1. Descripción general

Se desea modelar y simular un sistema automático de administración de medicación intravenosa. El sistema controla una bomba de infusión que debe suministrar una droga a un paciente respetando una dosis objetivo indicada por una orden médica.

En esta versión reducida del proyecto se considerará un controlador simplificado. El sistema debe recibir órdenes médicas, controlar el caudal administrado, detectar desviaciones persistentes respecto del caudal indicado y reaccionar ante la condición de fin de bolsa. También deberá gestionar alarmas asociadas a dichas situaciones.

El objetivo principal es estudiar el comportamiento temporal del sistema y verificar que las acciones de control ocurran dentro de los tiempos especificados.

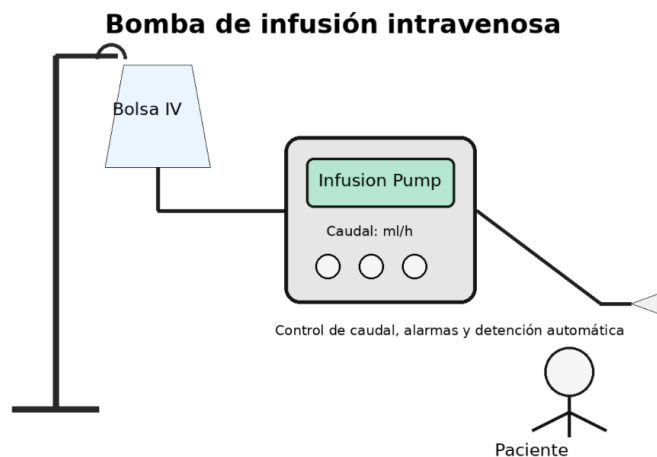


Figura 1: Esquema de una bomba de infusión intravenosa.

2. Objetivo del proyecto

El objetivo del proyecto es construir un modelo de simulación de eventos discretos que represente el comportamiento de una bomba de infusión simplificada. Se deben considerar escenarios normales de funcionamiento y escenarios de falla asociados al caudal administrado y al agotamiento de la bolsa de medicación.

3. Entradas del sistema

El sistema recibe los siguientes eventos de entrada:

- **ordenMedica:** indica el caudal objetivo que debe administrarse, expresado en ml/h. Las órdenes médicas podrán indicar caudales comprendidos entre 0 y 200 ml/h. Una orden con caudal igual a cero indica la detención de la infusión.
- **sensorFlujo:** informa periódicamente el caudal real administrado.
- **finBolsa:** indica que la bolsa de medicación está próxima a agotarse.
- **confirmacionEnfermero:** indica que una alarma fue reconocida por personal médico.

4. Salidas del sistema

Las salidas posibles del sistema son:

- **ajustarCaudal**: ordena modificar el caudal de infusión.
- **detenerBomba**: detiene la infusión.
- **alarmaBaja**: notificación preventiva o de baja criticidad.
- **alarmaMedia**: notificación por desviaciones sostenidas del funcionamiento esperado.
- **alarmaCritica**: notificación ante una condición que requiere atención inmediata.
- **registrarEvento**: registra el evento ocurrido en el registro de eventos del sistema.

5. Requisitos temporales

El sistema debe cumplir los siguientes requisitos temporales:

1. Luego de recibir una **ordenMedica** con caudal mayor que cero, la bomba debe comenzar la infusión en menos de 3 segundos.
2. Una orden médica especifica únicamente el caudal de infusión. La bomba mantiene dicho caudal hasta recibir una nueva **ordenMedica**. Una orden con caudal igual a cero indica la detención de la infusión.
3. El sensor de flujo debe informar el caudal real cada 1 segundo.
4. Si el caudal real difiere en más de un 10 % respecto del caudal indicado durante más de 5 segundos, el sistema debe emitir una **alarmaMedia**.
5. Si el sistema no logra corregir el desvío de caudal luego de emitir una **alarmaMedia**, podrá detener la bomba y emitir una **alarmaCritica**. El criterio exacto para pasar de **alarmaMedia** a **alarmaCritica** deberá ser definido por cada grupo.
6. Si una **alarmaCritica** no es confirmada por el enfermero dentro de 30 segundos, la alarma debe repetirse cada 10 segundos.
7. Si se detecta **finBolsa**, debe emitirse una **alarmaBaja** y la infusión sólo puede continuar durante un máximo de 60 segundos. Transcurridos los 60 segundos, la bomba debe detener automáticamente la infusión.
8. Luego de una **alarmaCritica**, la bomba permanecerá detenida hasta recibir una **confirmacionEnfermero** o una nueva **ordenMedica**.

6. Parámetros del sistema

Parámetro	Valor sugerido
Rango de caudal permitido	0 a 200 ml/h
Tiempo máximo para iniciar infusión	3 s
Período de muestreo del sensor de flujo	1 s
Latencia del actuador de la bomba	0.5 s
Desvío máximo permitido de caudal	10 %
Tiempo máximo de desvío de caudal	5 s
Tiempo para confirmar alarma crítica	30 s
Período de repetición de alarma crítica	10 s
Tiempo máximo luego de fin de bolsa	60 s

Cuadro 1: Parámetros temporales del sistema.

7. Componentes propuestos

El sistema puede modelarse mediante los siguientes modelos atómicos DEVS.

7.1. Generador de órdenes médicas

Genera eventos **ordenMedica** que indican el caudal objetivo que debe administrar la bomba. Las órdenes pueden generarse de manera determinística o estocástica, según los escenarios definidos por el grupo.

Salida: ordenMedica.

7.2. Sensor de flujo

Mide periódicamente el caudal real administrado por la bomba.

Entrada: caudalActual.

Salida: sensorFlujo.

Este sensor debe emitir una medición cada 1 segundo.

7.3. Controlador de bomba

Es el componente central del sistema. Recibe las órdenes médicas, las mediciones del sensor de flujo, la señal de fin de bolsa y las confirmaciones del enfermero. Decide si debe iniciar la infusión, ajustar el caudal, detener la bomba o emitir alarmas.

Entradas:

- ordenMedica
- sensorFlujo
- finBolsa
- confirmacionEnfermero

Salidas:

- ajustarCaudal
- detenerBomba
- alarmaBaja
- alarmaMedia
- alarmaCritica
- registrarEvento

7.4. Actuador de la bomba

Representa el mecanismo físico que administra la medicación al paciente. Recibe órdenes del controlador y modifica el caudal real.

Entradas: ajustarCaudal, detenerBomba.

Salida: caudalActual.

La latencia máxima para aplicar un ajuste de caudal es de 0.5 segundos.

7.5. Módulo de alarmas

Gestiona las alarmas visuales y sonoras. Debe repetir las alarmas críticas no confirmadas.

Entradas: alarmaBaja, alarmaMedia, alarmaCritica, confirmacionEnfermero.

Salida: notificacionAlarma.

8. Modelos atómicos DEVS

En esta sección se presentan los modelos atómicos del sistema utilizando una notación similar a CML-DEVS. Cada modelo se especifica mediante sus conjuntos de entrada, salida, estado, funciones de transición, función de salida y función de avance de tiempo.

8.1. Generador de órdenes médicas

atomic GeneradorOrdenes is $\langle Y, S, \delta_{int}, \lambda, ta \rangle$ where

S is

caudalObjetivo : [0,200];

σ : Time;

end S

Y is

out : {ordenMedica} x [0,200];

end Y

$\delta_{int}(\text{caudalObjetivo}, \sigma)$ is

caudalObjetivo = siguiente_caudal();

σ = siguiente_tiempo();

end δ_{int}

```

λ(caudalObjetivo, σ) is
    out = (ordenMedica, caudalObjetivo);
end λ

```

```

ta(caudalObjetivo, σ) is
    σ;
end ta

```

```

end atomic

```

Estado inicial: (100, 0).

$$S = (\text{caudalObjetivo}, \sigma)$$

donde *caudalObjetivo* es el valor de caudal que será enviado en la próxima orden médica y σ es el tiempo restante hasta la próxima generación de una orden médica.

8.2. Sensor de flujo

atomic SensorFlujo is <X, Y, S, δ_{int} , δ_{ext} , λ , ta> where

```

S is
    caudalReal : ℝ;
    σ : Time;
end S

```

```

X is
    in : {caudalActual} x ℝ;
end X

```

```

Y is
    out : {sensorFlujo} x ℝ;
end Y

```

```

δext((caudalReal, σ), e, (in,x)) is
    caudalReal = x.getSnd();
    σ = max(0, σ - e);
end δext

```

```

δint(caudalReal, σ) is
    σ = 1;
end δint

```

```

λ(caudalReal, σ) is
    out = (sensorFlujo, caudalReal);
end λ

```

```

ta(caudalReal, σ) is
    σ;
end ta

```

end atomic

Estado inicial: $(0,1)$.

$$S = (\text{caudalReal}, \sigma)$$

donde *caudalReal* es el caudal real administrado por la bomba y σ es la variable de avance del tiempo.

Además, se considera que `getFt()` devuelve el primer componente de una tupla y `getSnd()` devuelve el segundo componente de una tupla.

8.3. Controlador de bomba

atomic ControladorBomba is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, \text{ta} \rangle$ where

S is

```
fase : {0,1,2};
cObj  : [0,200];
cReal : [0,200];
eb    : {0,1};
ea    : {0,1,2,3};
td    : Time;
tb    : Time;
σ     : Time;
```

end S

X is

```
in : ({ordenMedica} x [0,200])
    ∪ ({sensorFlujo} x [0,200])
    ∪ {finBolsa, confirmacionEnfermero};
```

end X

Y is

```
out : ({ajustarCaudal} x [0,200])
      ∪ {detenerBomba, alarmaBaja, alarmaMedia, alarmaCritica};
```

end Y

$\delta_{ext}((\text{fase}, \text{cObj}, \text{cReal}, \text{eb}, \text{ea}, \text{td}, \text{tb}, \sigma), e, (\text{in}, x))$ is

defcases

case

```
fase = 1;
cObj = x.getSnd();
ea = 0;
td = 0;
σ = 0;
```

if

```
x.getFt() = ordenMedica and x.getSnd() > 0;
```

case

```
fase = 0;
cObj = 0;
ea = 0;
```



```

    td = 0;
     $\sigma$  = 0;
if
    x.getFt() = ordenMedica and x.getSnd() = 0;

case
    fase = 2;
    cReal = x.getSnd();
    td = td + e;
     $\sigma$  = max(0, 5 - td);
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) > 0.1 * cObj and fase
    = 2 and ea = 0;

case
    fase = 2;
    cReal = x.getSnd();
    td = 0;
     $\sigma$  = 5;
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) > 0.1 * cObj and fase
    != 2;

case
    fase = 1;
    cReal = x.getSnd();
    ea = 0;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) <= 0.1 * cObj;

case
    fase = 2;
    cReal = x.getSnd();
    td = td + e;
     $\sigma$  = max(0, 5 - td);
if
    x.getFt() = sensorFlujo and abs(x.getSnd() - cObj) > 0.1 * cObj and ea =
    2;

case
    eb = 1;
    ea = 1;
    tb = 0;
     $\sigma$  = 0;
if
    x = finBolsa;

case
    fase = 0;

```

```

    ea = 0;
     $\sigma$  =  $\infty$ ;
if
    x = confirmacionEnfermero and ea = 3 and fase = 0;

case
    ea = 0;
if
    x = confirmacionEnfermero;
end defcases
end  $\delta_{ext}$ 

 $\delta_{int}((\text{fase}, cObj, cReal, eb, ea, td, tb, \sigma))$  is
defcases
case
    fase = 1;
    ea = 0;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    fase = 1 and cObj > 0;

case
    fase = 0;
    cObj = 0;
    ea = 0;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    fase = 0 and cObj = 0;

case
    fase = 2;
    ea = 2;
    td = 0;
     $\sigma$  = 5;
if
    fase = 2 and ea = 0;

case
    fase = 0;
    ea = 3;
    td = 0;
     $\sigma$  =  $\infty$ ;
if
    fase = 2 and ea = 2;

case
    fase = fase;
    eb = 1;
    ea = 1;

```

```

        td = td;
        tb = 60;
         $\sigma$  = 60;
    if
        eb = 1 and ea = 1 and tb = 0;

    case
        fase = 0;
        cObj = 0;
        eb = 1;
        ea = 1;
        tb = 60;
         $\sigma$  =  $\infty$ ;
    if
        eb = 1 and ea = 1 and tb = 60;
    end defcases
end  $\delta_{int}$ 

 $\lambda((\text{fase}, \text{cObj}, \text{cReal}, \text{eb}, \text{ea}, \text{td}, \text{tb}, \sigma))$  is
defcases
    case
        out = alarmaBaja;
    if
        eb = 1 and ea = 1 and tb = 0;

    case
        out = detenerBomba;
    if
        eb = 1 and ea = 1 and tb = 60;

    case
        out = (ajustarCaudal, cObj);
    if
        fase = 1 and cObj > 0 and ea = 0;

    case
        out = detenerBomba;
    if
        fase = 0 and cObj = 0;

    case
        out = alarmaMedia;
    if
        fase = 2 and ea = 0;

    case
        out = alarmaCritica;
    if
        fase = 2 and ea = 2;
    end defcases
end  $\lambda$ 

```

```

ta((fase,cObj,cReal,eb,ea,td,tb,σ)) is
    σ;
end ta

```

```

end atomic

```

Estado inicial: $(0, 0, 0, 0, 0, 0, 0, \infty)$.

$$S = (fase, cObj, cReal, eb, ea, td, tb, \sigma)$$

donde:

- $fase = 0$: detenida.
- $fase = 1$: infundiendo.
- $fase = 2$: corrigiendo.
- $cObj$: caudal objetivo indicado por la última orden médica.
- $cReal$: caudal real informado por el sensor de flujo.
- $eb = 0$: bolsa normal.
- $eb = 1$: fin de bolsa.
- $ea = 0$: sin alarma.
- $ea = 1$: alarma baja.
- $ea = 2$: alarma media.
- $ea = 3$: alarma crítica.
- td : tiempo acumulado de desvío de caudal.
- tb : temporizador asociado a la condición de fin de bolsa.
- σ : tiempo de avance del modelo.

La fase *corrigiendo* representa el estado en el cual se detectó una desviación mayor al 10 % y el controlador se encuentra esperando confirmar si dicha desviación persiste para emitir una alarma media o crítica.

Si luego de emitir una **alarmaMedia** el desvío de caudal persiste durante otros 5 segundos, el controlador detiene la bomba y emite **alarmaCritica**.

8.4. Actuador de la bomba

atomic ActuadorBomba is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

```
S is
  d : [0,200];
  u : [0,200];
   $\sigma$  : Time;
end S

X is
  in : ({ajustarCaudal} x [0,200])  $\cup$  {detenerBomba};
end X

Y is
  out : {caudalActual} x [0,200];
end Y

 $\delta_{ext}((d,u,\sigma), e, (in,x))$  is
  defcases
    case
      u = 0;
       $\sigma$  = 0.5;
    if
      x.getFt() = detenerBomba;

    case
      u = x.getSnd();
       $\sigma$  = 0.5;
    if
      x.getFt() = ajustarCaudal;
    end defcases
  end  $\delta_{ext}$ 

 $\delta_{int}(d,u,\sigma)$  is
  d = u;
   $\sigma$  =  $\infty$ ;
end  $\delta_{int}$ 

 $\lambda(d,u,\sigma)$  is
  out = (caudalActual, u);
end  $\lambda$ 

ta(d,u, $\sigma$ ) is
   $\sigma$ ;
end ta

end atomic
```

Estado inicial: $(0, 0, \infty)$.

$$S = (d, u, \sigma)$$

donde d es el valor actual que está usando la bomba, u es el valor que llega desde el controlador y σ es la variable de tiempo de avance.

8.5. Módulo de alarmas

atomic ModuloAlarmas is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

S is

```
fase : {0,1,2,3};
tipo : {0,1,2,3};
 $\sigma$  : Time;
```

end S

X is

```
in : {alarmaBaja, alarmaMedia, alarmaCritica, confirmacionEnfermero};
```

end X

Y is

```
out : notificacionAlarma x {1,2,3};
```

end Y

$\delta_{ext}((fase, tipo, \sigma), e, (in, x))$ is

defcases

case

```
fase = 1;
tipo = 1;
 $\sigma$  = 0;
```

if

```
x = alarmaBaja;
```

case

```
fase = 1;
tipo = 2;
 $\sigma$  = 0;
```

if

```
x = alarmaMedia;
```

case

```
fase = 1;
tipo = 3;
 $\sigma$  = 0;
```

if

```
x = alarmaCritica;
```

case

```
fase = 0;
tipo = 0;
 $\sigma$  =  $\infty$ ;
```

if

```
x = confirmacionEnfermero;
```

end defcases

```

end  $\delta_{ext}$ 

 $\delta_{int}(fase, tipo, \sigma)$  is
defcases
  case
    fase = 0;
    tipo = 0;
     $\sigma = \infty$ ;
  if
    fase = 1 and tipo = 1;

  case
    fase = 0;
    tipo = 0;
     $\sigma = \infty$ ;
  if
    fase = 1 and tipo = 2;

  case
    fase = 2;
    tipo = 3;
     $\sigma = 30$ ;
  if
    fase = 1 and tipo = 3;

  case
    fase = 3;
    tipo = 3;
     $\sigma = 10$ ;
  if
    fase = 2 and tipo = 3;

  case
    fase = 3;
    tipo = 3;
     $\sigma = 10$ ;
  if
    fase = 3 and tipo = 3;
end defcases
end  $\delta_{int}$ 

 $\lambda(fase, tipo, \sigma)$  is
defcases
  case
    out = (notificacionAlarma, 1);
  if
    fase = 1 and tipo = 1;

  case
    out = (notificacionAlarma, 2);
  if

```

```

    fase = 1 and tipo = 2;

case
    out = (notificacionAlarma, 3);
if
    fase = 1 and tipo = 3;

case
    out = (notificacionAlarma, 3);
if
    fase = 2 and tipo = 3;

case
    out = (notificacionAlarma, 3);
if
    fase = 3 and tipo = 3;
end defcases
end λ

ta(fase,tipo,σ) is
    σ;
end ta

end atomic

```

Estado inicial: $(0, 0, \infty)$.

$$S = (fase, tipo, \sigma)$$

donde:

- $fase = 0$: pasivo.
- $fase = 1$: notificando.
- $fase = 2$: esperando confirmación.
- $fase = 3$: repitiendo.
- $tipo = 0$: ninguna.
- $tipo = 1$: baja.
- $tipo = 2$: media.
- $tipo = 3$: crítica.
- σ : tiempo de permanencia del estado.

8.6. Registro de eventos

atomic RegistroEventos is $\langle X, Y, S, \delta_{int}, \delta_{ext}, \lambda, ta \rangle$ where

S is

```
log : List Text;  
cantAlarmasBajas : N;  
cantAlarmasMedias : N;  
cantAlarmasCriticas : N;  
cantDetenciones : N;  
 $\sigma$  : Time;
```

end S

X is

```
in : {alarmaBaja, alarmaMedia, alarmaCritica, detenerBomba,  
      confirmacionEnfermero, finBolsa}  
     $\cup$  ({ajustarCaudal}  $\times$  [0,200]);
```

end X

Y is

```
out : {};
```

end Y

$\delta_{ext}((\log, \text{cantAlarmasBajas}, \text{cantAlarmasMedias}, \text{cantAlarmasCriticas}, \text{cantDetenciones},$
 $\sigma), e, (\text{in}, x))$ is

defcases

case

```
log = log  $\frown$  {"Alarma baja registrada"};  
cantAlarmasBajas = cantAlarmasBajas + 1;  
 $\sigma = \infty$ ;
```

if

```
x = alarmaBaja;
```

case

```
log = log  $\frown$  {"Alarma media registrada"};  
cantAlarmasMedias = cantAlarmasMedias + 1;  
 $\sigma = \infty$ ;
```

if

```
x = alarmaMedia;
```

case

```
log = log  $\frown$  {"Alarma critica registrada"};  
cantAlarmasCriticas = cantAlarmasCriticas + 1;  
 $\sigma = \infty$ ;
```

if

```
x = alarmaCritica;
```

case

```
log = log  $\frown$  {"Detencion de bomba registrada"};  
cantDetenciones = cantDetenciones + 1;  
 $\sigma = \infty$ ;
```

if

```
x = detenerBomba;
```

```

    case
        log = log  $\cup$  {"Ajuste de caudal registrado"};
         $\sigma = \infty$ ;
    if
        x.getFt() = ajustarCaudal;

    case
        log = log  $\cup$  {"Confirmacion de enfermero registrada"};
         $\sigma = \infty$ ;
    if
        x = confirmacionEnfermero;

    case
        log = log  $\cup$  {"Fin de bolsa registrado"};
         $\sigma = \infty$ ;
    if
        x = finBolsa;
    end defcases
end  $\delta_{ext}$ 

 $\delta_{int}((log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones,$ 
     $\sigma))$  is
     $\sigma = \infty$ ;
end  $\delta_{int}$ 

 $\lambda((log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones, \sigma)$ 
     $)$  is
    out = {};
end  $\lambda$ 

ta((log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones,  $\sigma$ 
     $))$  is
     $\sigma$ ;
end ta

end atomic

```

Estado inicial: ($[], 0, 0, 0, 0, \infty$).

$S = (log, cantAlarmasBajas, cantAlarmasMedias, cantAlarmasCriticas, cantDetenciones, \sigma)$

donde:

- *log*: lista de textos que representa el registro histórico de eventos ocurridos.
- *cantAlarmasBajas*: cantidad de alarmas bajas registradas.
- *cantAlarmasMedias*: cantidad de alarmas medias registradas.
- *cantAlarmasCriticas*: cantidad de alarmas críticas registradas.

- $cantDetenciones$: cantidad de detenciones de bomba registradas.
- σ : tiempo de avance del modelo. Permanece en ∞ porque el registrador sólo reacciona ante eventos externos.

9. Modelo acoplado

El modelo acoplado representa la integración de los modelos atómicos definidos anteriormente. En este sistema, el **Controlador de bomba** actúa como componente central, recibiendo las órdenes médicas, las mediciones del sensor de flujo, la señal de fin de bolsa y las confirmaciones del enfermero.

A partir de estos eventos, el controlador decide si debe ajustar el caudal, detener la bomba o emitir alarmas. El **Actuador de la bomba** modifica el caudal real administrado, mientras que el **Sensor de flujo** mide periódicamente dicho caudal y lo informa nuevamente al controlador.

Además, el **Módulo de alarmas** gestiona las notificaciones generadas y el **Registro de eventos** almacena los eventos relevantes del sistema.

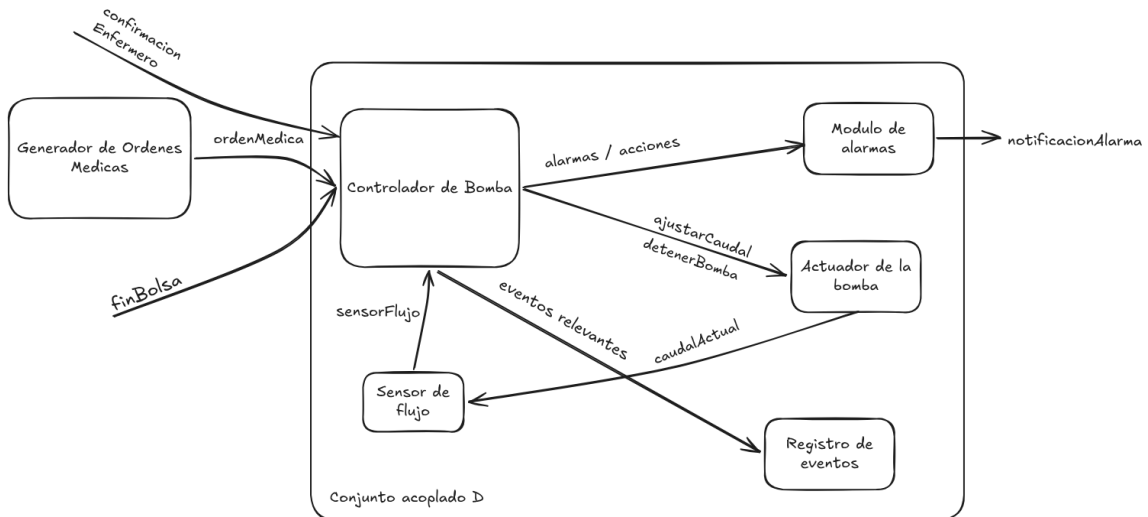


Figura 2: Modelo acoplado del sistema simplificado de bomba de infusión.

9.1. Especificación formal del acoplamiento

El modelo acoplado se define como:

$$N = \langle X_N, Y_N, D, \{M_d\}, EIC, EOC, IC, Select \rangle$$

donde N representa el sistema completo de bomba de infusión simplificada.

$$D = \{ControladorBomba, SensorFlujo, ActuadorBomba, ModuloAlarmas, RegistroEventos\}$$

Los modelos contenidos en el conjunto D corresponden a los componentes internos del sistema acoplado. El *Generador de Órdenes Médicas* se considera parte del entorno de simulación y no pertenece al modelo acoplado, por lo que su salida ingresa a N como una entrada externa.

Entradas y salidas externas

Las entradas externas del sistema son:

$$X_N = \{pOrdenMedica, pFinBolsa, pConfirmacionEnfermero\}$$

La salida externa formal del sistema es:

$$Y_N = \{pNotificacionAlarma\}$$

El componente *RegistroEventos* no se incluye como salida externa formal del modelo acoplado, ya que su función es mantener internamente un registro histórico o *log* de los eventos ocurridos durante la simulación. Dicho registro puede consultarse luego para el análisis de resultados, pero no se modela como un evento de salida DEVS.

Acoplamientos de entrada externa

Los acoplamientos de entrada externa, *EIC*, conectan las entradas del modelo acoplado *N* con los componentes internos correspondientes:

$$EIC = \{[(N, pOrdenMedica), (ControladorBomba, pOrdenMedica)]; \\ [(N, pFinBolsa), (ControladorBomba, pFinBolsa)]; \\ [(N, pFinBolsa), (RegistroEventos, pFinBolsa)]; \\ [(N, pConfirmacionEnfermero), (ControladorBomba, pConfirmacionEnfermero)]; \\ [(N, pConfirmacionEnfermero), (ModuloAlarmas, pConfirmacionEnfermero)]; \\ [(N, pConfirmacionEnfermero), (RegistroEventos, pConfirmacionEnfermero)]\}$$

La orden médica generada por el entorno de simulación ingresa al sistema acoplado y se envía al controlador. La señal de fin de bolsa se envía al controlador para que tome la acción correspondiente y también al registro de eventos para guardar dicha ocurrencia. La confirmación del enfermero se envía al controlador, al módulo de alarmas y al registro de eventos.

Acoplamientos de salida externa

Los acoplamientos de salida externa, *EOC*, conectan las salidas de los modelos internos con las salidas externas del sistema acoplado:

$$EOC = \{[(ModuloAlarmas, pNotificacionAlarma), (N, pNotificacionAlarma)]\}$$

La salida externa del sistema corresponde a las notificaciones emitidas por el módulo de alarmas.

Acoplamientos internos

Los acoplamientos internos, *IC*, conectan las salidas de los modelos atómicos con las entradas de otros modelos atómicos dentro del sistema acoplado:

$$\begin{aligned}
IC = \{ & [(\text{ControladorBomba}, p\text{AjustarCaudal}), (\text{ActuadorBomba}, p\text{AjustarCaudal})]; \\
& [(\text{ControladorBomba}, p\text{DetenerBomba}), (\text{ActuadorBomba}, p\text{DetenerBomba})]; \\
& [(\text{ControladorBomba}, p\text{AlarmaBaja}), (\text{ModuloAlarmas}, p\text{AlarmaBaja})]; \\
& [(\text{ControladorBomba}, p\text{AlarmaMedia}), (\text{ModuloAlarmas}, p\text{AlarmaMedia})]; \\
& [(\text{ControladorBomba}, p\text{AlarmaCritica}), (\text{ModuloAlarmas}, p\text{AlarmaCritica})]; \\
& [(\text{ActuadorBomba}, p\text{CaudalActual}), (\text{SensorFlujo}, p\text{CaudalActual})]; \\
& [(\text{SensorFlujo}, p\text{SensorFlujo}), (\text{ControladorBomba}, p\text{SensorFlujo})]; \\
& [(\text{ControladorBomba}, p\text{AjustarCaudal}), (\text{RegistroEventos}, p\text{AjustarCaudal})]; \\
& [(\text{ControladorBomba}, p\text{DetenerBomba}), (\text{RegistroEventos}, p\text{DetenerBomba})]; \\
& [(\text{ControladorBomba}, p\text{AlarmaBaja}), (\text{RegistroEventos}, p\text{AlarmaBaja})]; \\
& [(\text{ControladorBomba}, p\text{AlarmaMedia}), (\text{RegistroEventos}, p\text{AlarmaMedia})]; \\
& [(\text{ControladorBomba}, p\text{AlarmaCritica}), (\text{RegistroEventos}, p\text{AlarmaCritica})] \}
\end{aligned}$$

El controlador recibe las mediciones periódicas del sensor de flujo y, a partir de ellas, puede emitir órdenes de ajuste de caudal o detención de la bomba. El actuador modifica el caudal real administrado y comunica dicho caudal al sensor de flujo. Luego, el sensor envía la medición al controlador.

Las alarmas generadas por el controlador son enviadas al módulo de alarmas para su gestión. Además, las acciones de ajuste de caudal, detención de bomba y emisión de alarmas son enviadas al modelo *RegistroEventos*, encargado de mantener el historial interno de funcionamiento del sistema.

Función de desempate

Para resolver posibles eventos simultáneos entre componentes, se define la siguiente prioridad:

$$\text{Select} = \left\langle \begin{array}{l} \text{ControladorBomba,} \\ \text{ActuadorBomba,} \\ \text{SensorFlujo,} \\ \text{ModuloAlarmas,} \\ \text{RegistroEventos} \end{array} \right\rangle$$

Esta prioridad favorece primero la toma de decisiones del controlador, luego la ejecución física de las acciones mediante el actuador, la actualización de las mediciones del sensor, la gestión de alarmas y finalmente el registro de eventos.